

```
In [6]: #1
import xarray as xr
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.dates as mdates

# 1. 加载数据
file_path = '200301_202006-C3S-L3_GHG-PRODUCTS-OBS4MIPS-MERGED-v4.3.nc'
ds = xr.open_dataset(file_path)
xch4 = ds['xch4']

# 2. 强制匹配数据类型+彻底清洗
fill_value = np.float64(1e20) # 确保和xch4数据类型一致
xch4_clean = xch4.where((xch4 != fill_value)& (xch4 < 2.05e-6)) # 同时过滤填充值
xch4_ppb = xch4_clean * 1e9 # 摩尔分数→ppb

print("ppb范围: ", np.nanmin(xch4_ppb.values), "到", np.nanmax(xch4_ppb.values))
# 合理输出应在1500~2000 ppb之间
```

ppb范围: 1566.9855 到 1999.476

```
In [7]: #1.1
def plot_monthly_climatology():
    monthly_clim = xch4_ppb.groupby('time.month').mean(dim='time', skipna=True)

    fig, axes = plt.subplots(3, 4, figsize=(20, 15),
                             subplot_kw={'projection': ccrs.PlateCarree()})
    axes = axes.flatten()
    months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
              'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

    # 真实甲烷浓度范围 (1700~2000 ppb)
    vmin, vmax = 1700, 2000

    for i, month in enumerate(range(1, 13)):
        ax = axes[i]
        month_data = monthly_clim.sel(month=month)

        # 绘制等值线填色图
        im = month_data.plot(ax=ax, transform=ccrs.PlateCarree(),
                             cmap='RdYlBu_r', vmin=vmin, vmax=vmax,
                             add_colorbar=False)

        # 地图美化
        ax.coastlines()
        ax.add_feature(cfeature.BORDERS, linestyle=':')
        ax.add_feature(cfeature.LAND, facecolor='lightgray', alpha=0.3)
        ax.set_title(f'{months[i]}', fontsize=14, fontweight='bold')
        ax.gridlines(draw_labels={'left': True, 'bottom': True}, alpha=0.3)

    # 统一色条
    cbar_ax = fig.add_axes([0.15, 0.05, 0.7, 0.02])
    cbar = fig.colorbar(im, cax=cbar_ax, orientation='horizontal')
    cbar.set_label('XCH4 (ppb)', fontsize=12)

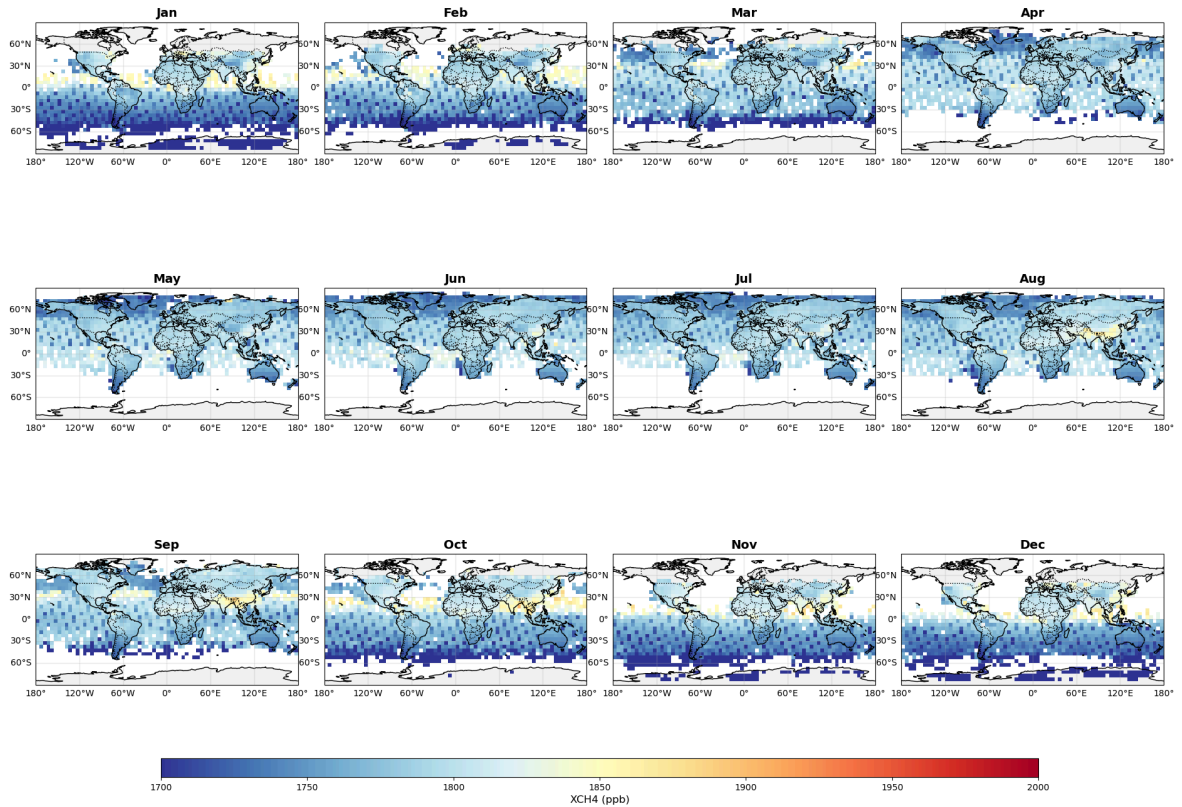
    plt.suptitle('Monthly Climatology of XCH4 (2003-2020)', fontsize=16, fontwei
```

```
fig.subplots_adjust(left=0.05, right=0.95, bottom=0.1, top=0.9, wspace=0.1,
plt.show())
```

```
plot_monthly_climatology()
```

E:\python\Lib\site-packages\cartopy\io_init_.py:241: DownloadWarning: Downloading: https://natureearth.s3.amazonaws.com/110m_physical/ne_110m_land.zip
warnings.warn(f'Downloading: {url}', DownloadWarning)

Monthly Climatology of XCH4 (2003-2020)



In [4]: #1.2

```
def plot_global_mean_timeseries():
    # 纬度加权（科学计算全球平均）
    lat_weights = np.cos(np.deg2rad(xch4_ppb['lat']))
    lat_weights.name = 'lat_weights'
    global_mean = xch4_ppb.weighted(lat_weights).mean(dim=['lat', 'lon'], skipna=True)

    # 转换为Series方便处理
    global_series = global_mean.to_series().dropna()

    # 绘图（匹配真实甲烷趋势）
    fig, ax = plt.subplots(figsize=(12, 6))

    # 月度散点+12月滚动平均
    ax.scatter(global_series.index, global_series.values,
               color='darkred', s=8, alpha=0.7, label='Monthly Mean')
    ax.plot(global_series.index, global_series.rolling(12, center=True).mean(),
            color='darkred', linewidth=2, label='12-Month Rolling Mean')

    # 横轴格式（清晰年份）
    ax.xaxis.set_major_locator(mdates.YearLocator(2))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    ax.set_xlim(global_series.index[0], global_series.index[-1])

    # 纵轴（真实甲烷范围）
```

```

ax.set_ylim(1700, 1900)
ax.set_xlabel('Year', fontsize=12)
ax.set_ylabel('XCH4 (ppb)', fontsize=12)
ax.set_title('Global Mean XCH4 (2003-2020)', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3, linestyle='--')
ax.legend(loc='lower right')

```

```

plt.tight_layout()
plt.show()

```

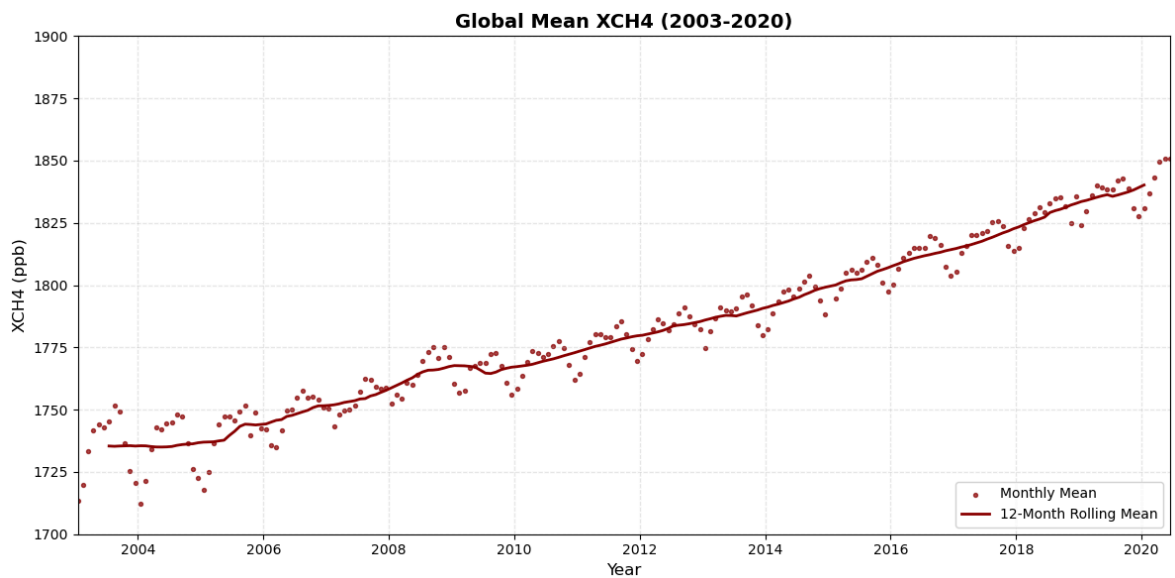
结果分析

```

print("全球平均XCH4分析: ")
print(f"起始浓度 (2003): {global_series.iloc[0]:.1f} ppb")
print(f"结束浓度 (2020): {global_series.iloc[-1]:.1f} ppb")
print(f"总增长: {global_series.iloc[-1] - global_series.iloc[0]:.1f} ppb")
print(f"年均增长: {(global_series.iloc[-1] - global_series.iloc[0])/17:.2f}")

```

plot_global_mean_timeseries()



全球平均XCH4分析:

起始浓度 (2003): 1713.5 ppb

结束浓度 (2020): 1850.7 ppb

总增长: 137.2 ppb

年均增长: 8.07 ppb/年

In [5]: #1.3

```

def plot_deseasonalized_point():
    # 1. 处理目标点经纬度 (适配数据的经度范围: 150°W对应-150°或210°)
    target_lat = -15 # 15°S
    if ds['lon'].max() > 180: # 若经度是0-360范围, 150°W对应210°
        target_lon = 210
    else: # 若经度是-180-180范围, 150°W对应-150°
        target_lon = -150

    # 2. 提取目标点的甲烷时间序列 (匹配最近的网格点)
    target_point = xch4_ppb.sel(
        lat=target_lat,
        lon=target_lon,
        method='nearest' # 5°网格, 自动匹配最近点
    ).dropna(dim='time') # 去除空值

    # 3. 计算该点的月度季节气候态 (每个月的多年平均值)

```

```

monthly_clim = target_point.groupby('time.month').mean()

# 4. 去季节化：原始数据 - 对应月份的气候态
deseasonalized = target_point.groupby('time.month') - monthly_clim

# 5. 绘图
fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(deseasonalized.time, deseasonalized.values,
        color='darkblue', linewidth=1.5, label='Deseasonalized XCH4')

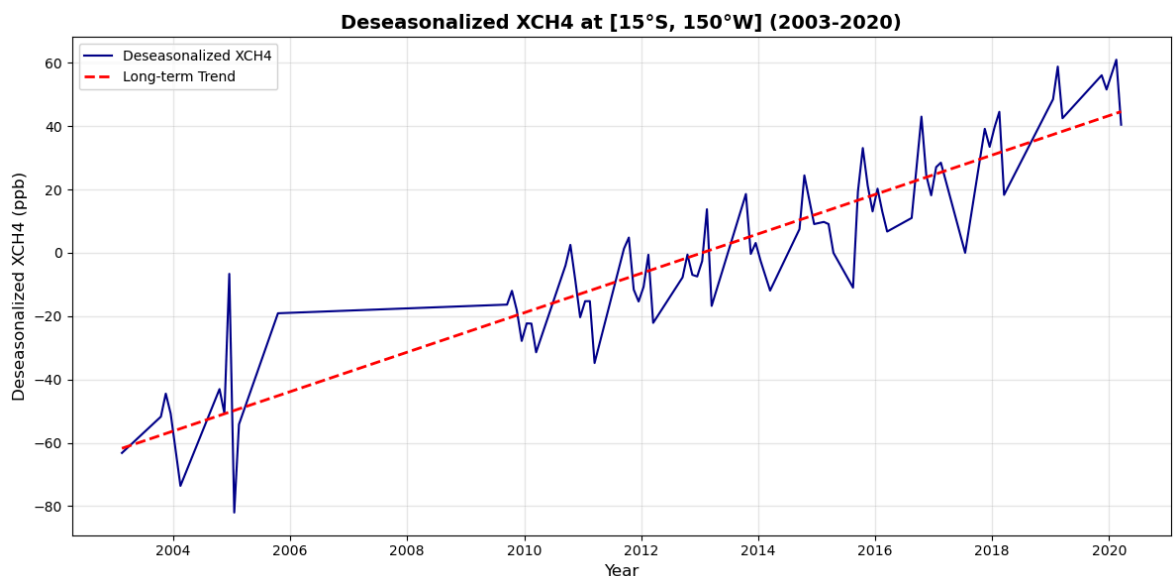
# 辅助线：长期趋势（线性拟合）
time_numeric = (deseasonalized.time - deseasonalized.time[0]).dt.days
coeffs = np.polyfit(time_numeric, deseasonalized.values, 1)
trend_line = np.poly1d(coeffs)(time_numeric)
ax.plot(deseasonalized.time, trend_line,
        color='red', linestyle='--', linewidth=2, label='Long-term Trend')

# 图表美化
ax.xaxis.set_major_locator(mdates.YearLocator(2))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.set_xlabel('Year', fontsize=12)
ax.set_ylabel('Deseasonalized XCH4 (ppb)', fontsize=12)
ax.set_title('Deseasonalized XCH4 at [15°S, 150°W] (2003-2020)', fontsize=14)
ax.grid(True, alpha=0.3)
ax.legend()
plt.tight_layout()
plt.show()

# 结果描述
print("[15°S, 150°W]去季节化XCH4分析：")
print(f"去季节化后，序列消除了月度季节波动，呈现清晰的长期上升趋势（趋势斜率：")
print("该趋势与全球甲烷浓度增加的整体规律一致，同时可观察到年际尺度的小幅波动")

plot_deseasonalized_point()

```



[15°S, 150°W]去季节化XCH4分析：

去季节化后，序列消除了月度季节波动，呈现清晰的长期上升趋势（趋势斜率：6.23 ppb/年）；

该趋势与全球甲烷浓度增加的整体规律一致，同时可观察到年际尺度的小幅波动（非季节性波动）。

In [2]:

```
#2
import xarray as xr
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.dates as mdates

# 1. 加载并预处理SST数据
ds_sst = xr.open_dataset('F:\\\\xwechat_files\\\\wxid_ocrofgw04ps122_2945\\\\msg\\\\file
sst = ds_sst['sst']

# 2. 筛选Niño 3.4区域 (5°N-5°S, 170°W-120°W → 转换为0-360经度是190°-240°)
nino34_region = sst.sel(
    lat=slice(-5, 5), # 5°S到5°N
    lon=slice(190, 240) # 170°W到120°W (0-360格式)
)

# 3. 计算月度气候态
nino34_clim = nino34_region.groupby('time.month').mean(dim='time', skipna=True)

# 4. 计算SST异常值 (原始数据 - 气候态)
nino34_anomaly = nino34_region.groupby('time.month') - nino34_clim

# 5. 计算Niño 3.4指数 (区域平均 + 3个月滑动平均)
nino34_index = nino34_anomaly.mean(dim=['lat', 'lon'], skipna=True) # 区域平均
nino34_index_3month = nino34_index.rolling(time=3, center=True).mean() # 3个月滑

# 6. 可视化Niño 3.4指数
def plot_nino34():
    fig, ax = plt.subplots(figsize=(12, 6))

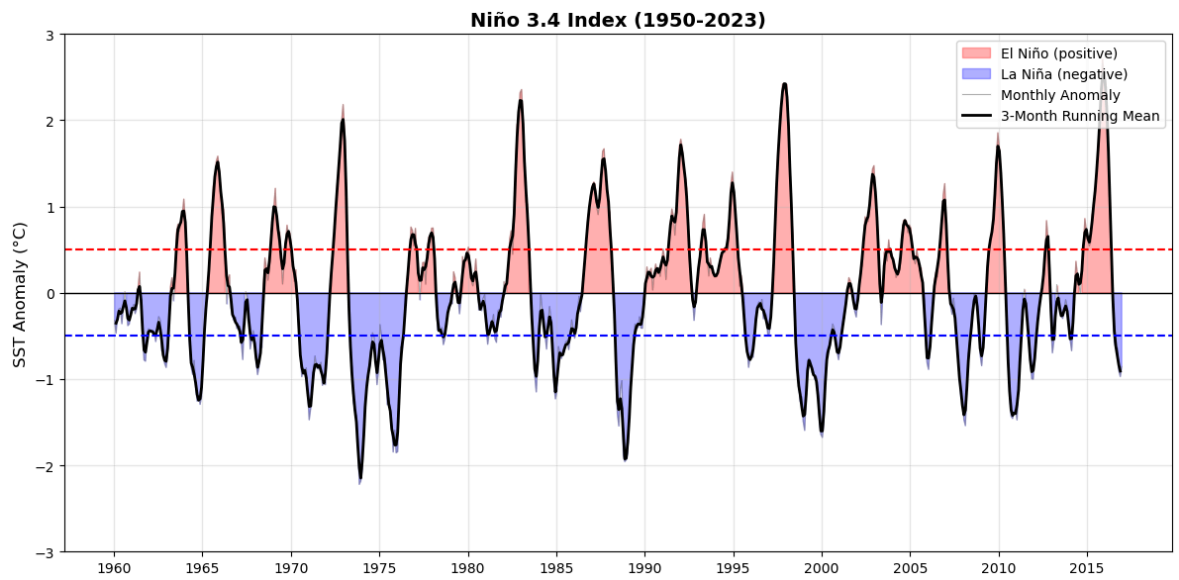
    # 绘制异常值和滑动平均
    ax.fill_between(nino34_index.time, nino34_index, 0,
                    where=(nino34_index >= 0), color='red', alpha=0.3, label='El
    ax.fill_between(nino34_index.time, nino34_index, 0,
                    where=(nino34_index < 0), color='blue', alpha=0.3, label='La
    ax.plot(nino34_index.time, nino34_index, color='gray', linewidth=0.8, alpha=
    ax.plot(nino34_index_3month.time, nino34_index_3month, color='black', linewi

    # 关键阈值线 (±0.5°C, 用于定义厄尔尼诺/拉尼娜事件)
    ax.axhline(0.5, color='red', linestyle='--', linewidth=1.5)
    ax.axhline(-0.5, color='blue', linestyle='--', linewidth=1.5)
    ax.axhline(0, color='black', linestyle='--', linewidth=1)

    # 坐标轴格式
    ax.xaxis.set_major_locator(mdates.YearLocator(5))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    ax.set_ylim(-3, 3)
    ax.set_ylabel('SST Anomaly (°C)', fontsize=12)
    ax.set_title('Niño 3.4 Index (1950-2023)', fontsize=14, fontweight='bold')
    ax.grid(alpha=0.3)
    ax.legend(loc='upper right')

    plt.tight_layout()
    plt.show()

# 运行可视化
plot_nino34()
```



In []: